

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Ethical Hacking

COURSE CODE: ITA14

COURSE OUTCOME COVERED

CO3: Analyze network services using port scanning, ping sweeps, and enumeration techniques across different operating systems to identify vulnerabilities. (BT4)
Assessment Tool Weightage: 30%

Duration: 90 min Total Marks: 50

Port Scanning Challenge

1. How would you identify and prioritize vulnerabilities after discovering multiple open ports during a port scan? Explain your approach.
2. A SYN scan and a TCP Connect scan produce different results on the same target. What factors could cause this difference, and how would you verify the findings?
3. During a port scan, most ports appear as "filtered." What security mechanisms might be responsible, and what additional techniques would you use to gather more information?
4. Compare TCP, UDP, and stealth (SYN) port scanning techniques. In which scenarios would each be the most effective, and what are their advantages and limitations?
5. Suppose you are conducting an authorized security assessment and discover critical services (such as SSH, FTP, and database ports) exposed to the internet. How would you assess the risks and recommend mitigation strategies?

Code of Conduct:

I _____Aluri Poojitha_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Poojitha

Question 1

How would you identify and prioritize vulnerabilities after discovering multiple open ports during a port scan? Explain your approach.

Answer

Once a scan returns a list of open ports, the goal shifts from discovery to risk-based triage. A structured approach has four stages:

- 1. Service and version fingerprinting:** Run a version-detection scan (e.g., `nmap -sV -sC`) on every open port to identify the exact service, daemon, and version running behind it. Banner grabbing and default-page inspection help confirm what nmap reports, since some administrators change default ports to obscure services.
- 2. Vulnerability correlation:** Match each identified service/version against known-vulnerability databases such as NVD/CVE, Exploit-DB, and vendor security advisories. Automated tools like Nessus, OpenVAS, or nmap's `vuln/vulners` NSE scripts can cross-reference banners with CVE entries and flag outdated or end-of-life software.
- 3. Risk scoring:** Prioritize findings using a combination of exploitability and impact rather than treating every open port equally. CVSS base scores give a starting point, but they should be weighted by exposure (internet-facing vs internal), presence of a public exploit or working PoC, authentication requirements, and the business criticality of the asset. A CVSS 9+ vulnerability with a public exploit on an internet-facing database port outranks a CVSS 9 on an internal, firewalled test host.
- 4. Contextual prioritization:** Group findings by attack surface — administrative interfaces (SSH, RDP, Telnet), data services (SQL, FTP, SMB), and web services — since compromise of an admin interface typically yields greater lateral-movement potential than a low-value informational port. Ports allowing anonymous or default-credential access are escalated regardless of raw CVSS score, because they require no exploit development at all.

The end output should be a ranked remediation list (Critical → High → Medium → Low) with a clear justification for each ranking, so that patching or mitigation effort is spent first on the issues that materially reduce the greatest risk.

Question 2

A SYN scan and a TCP Connect scan produce different results on the same target. What factors could cause this difference, and how would you verify the findings?

Answer

A SYN scan (half-open scan) sends a SYN packet and evaluates the response without completing the three-way handshake, while a TCP Connect scan uses the operating system's `connect()` call to complete the full handshake. Because they operate at different layers, several factors can make their results diverge:

Firewall and IDS/IPS behavior: Some firewalls are configured to drop or rate-limit raw SYN packets typical of stealth scans while allowing legitimate, fully-established connections through, so a SYN scan may report a port as filtered while a Connect scan on the same port succeeds (or vice versa if the firewall specifically inspects completed sessions).

Privilege level of the scanning tool: SYN scanning requires raw-socket access (root/administrator privileges). If the scanner lacks these privileges it silently falls back to a Connect scan, producing a different packet pattern than intended and skewing comparison between two runs.

Application-layer filtering: A Connect scan completes the handshake and may trigger the application to respond, reset, or drop the connection based on payload inspection, request rate, or connection limits — behavior a half-open SYN probe never reaches.

Packet loss, rate limiting, and timing: SYN scans are typically faster and send more packets per second, which can trigger rate-limiting or IDS-based dropping that a slower Connect scan avoids, and network congestion or timeouts can cause open ports to be misreported as filtered in either scan.

Stateful load balancers/NAT devices: Devices that track full TCP sessions may respond differently to half-open attempts than to completed connections, especially in cloud or load-balanced environments.

To verify which result is accurate, I would: (1) re-run both scans multiple times with adjusted timing templates (e.g., -T2 instead of -T4) to rule out rate-limiting artifacts; (2) run the scans from a different network vantage point or IP to rule out source-based filtering; (3) perform a manual verification with a tool like netcat or telnet to attempt an actual application-layer connection and observe the banner/response directly; (4) capture traffic with Wireshark/tcpdump during both scans to see exactly how the target responds (RST, no response, SYN-ACK) at the packet level; and (5) cross-check with an alternate scanner (e.g., masscan or a different nmap scan type such as -sA for ACK) to see whether the discrepancy is tool-specific or genuinely reflects the target's filtering policy.

Question 3

During a port scan, most ports appear as "filtered." What security mechanisms might be responsible, and what additional techniques would you use to gather more information?

Answer

A "filtered" result means the scanner could not determine whether the port is open or closed because probe packets were dropped or blocked, rather than answered with a definitive SYN-ACK or RST.

Likely security mechanisms

- Perimeter or host-based firewalls configured to silently drop (rather than reject) unsolicited packets, which is standard practice to avoid revealing port state to attackers.
- Intrusion Prevention/Detection Systems (IPS/IDS) that detect scanning patterns and begin dropping packets from the offending source IP after a threshold is crossed.
- Access Control Lists (ACLs) on routers/switches restricting traffic to specific source IPs, subnets, or ports.
- Rate limiting or connection throttling on the target or an upstream device, which causes probes to time out rather than receive a response.
- Network Address Translation (NAT) or a stateful firewall that only permits packets belonging to already-established, internally-initiated sessions.
- Endpoint protection / host firewalls (e.g., Windows Defender Firewall, iptables) configured with a default DROP policy for unrecognized inbound traffic.

Techniques to gather more information

- 1. Alternate scan types:** Use ACK scans (-sA) to map firewall rule sets and distinguish stateful vs stateless filtering, FIN/NULL/Xmas scans to test how the target's TCP stack handles unusual flag combinations (useful against simple packet filters, though modern stateful firewalls typically block these too), and UDP scans if TCP is heavily filtered but the service may run over UDP.
- 2. Timing and decoy adjustments:** Slow the scan (-T1/-T2), fragment packets (-f), or use decoy IPs (-D) to evade signature-based IDS/IPS detection thresholds that may be causing the drops.
- 3. Source port and firewall-evasion tricks:** Spoof the source port to a commonly-trusted value (e.g., 53 or 80) to see if a misconfigured ACL is fooled into allowing the traffic through.
- 4. Traceroute/path analysis:** Use traceroute or nmap's --traceroute to identify at which hop packets stop responding, helping pinpoint whether filtering happens at the perimeter, an intermediate device, or the host itself.
- 5. Out-of-band verification:** Where authorized, check with the system/network administrator or internal documentation to confirm expected firewall policy, and correlate scan results with any available firewall or IDS logs.
- 6. Passive reconnaissance:** Supplement active scanning with passive techniques (DNS records, certificate transparency logs, Shodan/Censys lookups) to infer which services are likely running without directly probing the filtered host.

Question 4

Compare TCP, UDP, and stealth (SYN) port scanning techniques. In which scenarios would each be the most effective, and what are their advantages and limitations?

Answer

Aspect	TCP Connect Scan	UDP Scan	Stealth (SYN) Scan
Mechanism	Completes full 3-way handshake via OS connect()	Sends UDP datagram; relies on ICMP port-unreachable or service response	Sends SYN only; tears down before handshake completes
Speed	Slower — full connection overhead per port	Slow — often needs retransmits and ICMP wait times	Fast — minimal packets, no full connection
Stealth	Low — logged as a completed connection by the OS/app	Low-medium — some logging, but less common target	High — rarely logged as a full connection ('half-open')
Privilege needed	None (standard sockets)	Root/admin for raw sockets (recommended)	Root/admin (raw sockets required)
Best used when	No raw-socket privileges available; need highest accuracy on TCP services	Enumerating services like DNS(53), SNMP(161), DHCP — non-TCP protocols	Authorized pentests needing speed and reduced detection footprint
Limitations	Easily logged/detected; slower on large ranges	Very slow, unreliable when ICMP is rate-limited/blocked; hard to distinguish open vs filtered	Requires elevated privileges; some IDS/IPS still detect the pattern

When each is most effective

TCP Connect scan: Best when the scanner cannot obtain raw-socket privileges (e.g., running as a non-root/non-admin user), or when maximum accuracy on TCP service state is required and stealth is not a concern — for example, a sanctioned internal audit where detection doesn't matter.

UDP scan: Essential when auditing services that run over UDP — DNS, SNMP, DHCP, NTP, TFTP — which a TCP-only scan would completely miss. It's the right choice whenever the target is suspected of running UDP-based infrastructure services, even though it is markedly slower and less definitive.

Stealth (SYN) scan: The default choice for authorized penetration tests where speed and reduced logging footprint matter — scanning large address ranges quickly while minimizing the connection-level noise picked up by target logs or basic IDS.

In practice, a thorough assessment combines all three: SYN scanning for fast, broad TCP discovery, a targeted UDP scan against known UDP service ports, and occasional Connect-scan verification when raw-socket scans give ambiguous (filtered) results.

Question 5

Suppose you are conducting an authorized security assessment and discover critical services (such as SSH, FTP, and database ports) exposed to the internet. How would you assess the risks and recommend mitigation strategies?

Answer

Risk assessment approach

- 1. Confirm exposure and necessity:** Verify the service is genuinely reachable from the public internet (not just from an internal test point) and determine whether internet exposure is actually required for business purposes, since many such exposures are unintentional misconfigurations.
- 2. Enumerate service details:** Identify exact software versions (OpenSSH, vsftpd/ProFTPD, MySQL/PostgreSQL/MSSQL, etc.), supported authentication mechanisms, and whether default or weak credentials are in use. Check for known CVEs against the identified versions.
- 3. Evaluate authentication and encryption posture:** FTP transmits credentials and data in plaintext, making it high-risk on the open internet; check whether SSH allows password authentication (vs key-based only) and whether outdated protocol versions or weak ciphers are enabled; check whether the database accepts remote connections with default accounts or without TLS.
- 4. Assess business impact:** Determine what data or systems each service exposes if compromised — for a database, this could mean direct access to sensitive records; for SSH/FTP, it could mean full host compromise or a pivot point into the internal network.
- 5. Assign a risk rating:** Combine likelihood (ease of exploitation, presence of weak credentials, known exploits) with impact (data sensitivity, criticality of the host) to produce a rating such as Critical/High/Medium/Low for each exposed service.

Recommended mitigation strategies

Remove unnecessary exposure: Where internet access isn't a genuine business requirement, place the service behind a VPN, bastion host, or restrict access via firewall/ACL to specific known IP ranges rather than 0.0.0.0/0.

Harden authentication: Enforce key-based authentication and disable password login for SSH; disable anonymous FTP and migrate to SFTP/FTPS for encrypted transfer; enforce strong, unique credentials and multi-factor authentication wherever supported; disable or rename default database accounts.

Patch and update: Upgrade all identified services to the latest stable, supported versions to close known CVEs; establish a regular patch-management cadence.

Network segmentation: Move database servers off the public-facing network entirely, placing them in a private subnet accessible only to application servers that require them.

Encrypt in transit: Enforce TLS/SSL for database connections, and disable legacy unencrypted protocols wherever an encrypted alternative exists.

Monitoring and rate limiting: Deploy fail2ban or equivalent intrusion-prevention to block repeated failed authentication attempts, enable centralized logging/alerting for the exposed services, and use rate limiting to slow brute-force attempts.

Principle of least privilege: Restrict database accounts and SSH users to only the minimum permissions/commands required for their role, and audit exposed services periodically as part of ongoing vulnerability management rather than a one-time fix.

Together, these steps reduce the attack surface (by removing or restricting unnecessary exposure), raise the cost of exploitation (through hardened authentication and patching), and improve detection/response capability (through logging and monitoring) — addressing the risk from prevention, containment, and detection angles simultaneously.